



Introduction to Artificial Intelligence

Sangyeon Kim

Ph.D

강의계획표

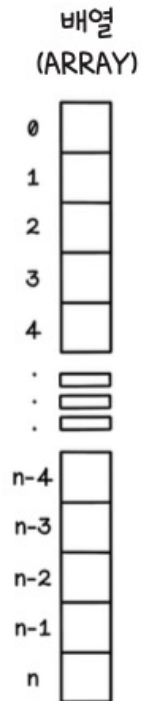
1. 인공지능 기초 개념과 역사
2. 지능형 에이전트
3. **탐색의 개념과 기초**
4. 탐색을 통한 문제해결
5. 복잡한 환경 탐색
6. 적대적 및 게임 탐색
7. 주요 탐색 기법 정리
8. Mid-term
9. 기계 학습 기초
10. 선형 회귀
11. 결정 트리
12. 인공신경망
13. 딥러닝
14. 강화학습
15. Final-term

탐색(Search)?

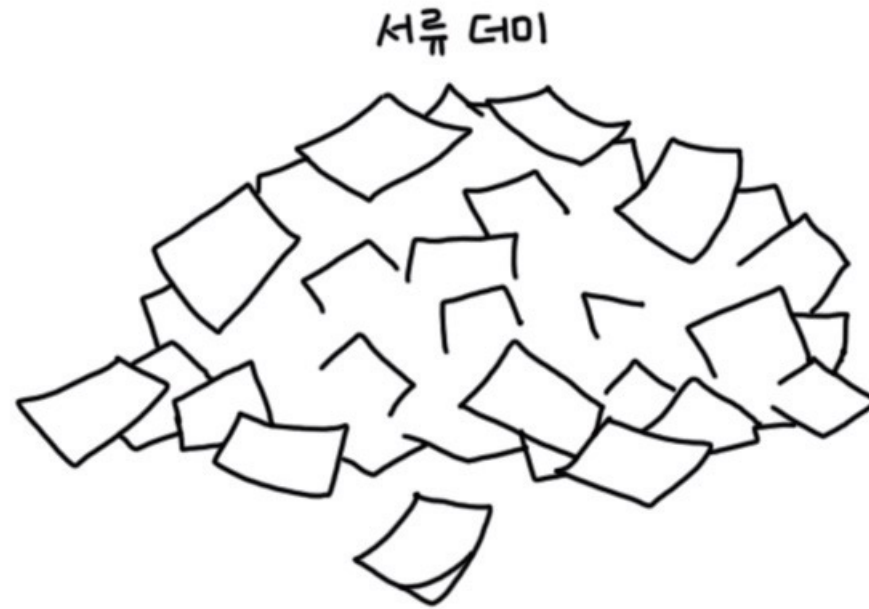
- 탐색(Search)은 원하는 것을 찾거나 목표 도달에 도움이 되는 결과를 추구하는 과정, 즉 목표를 향해 가는 지능적 활동을 의미함
- 컴퓨터과학 분야에서의 탐색은 주로 주어진 데이터 집합에서 원하는 데이터를 찾는 과정을 의미하며, 많은 탐색 기법이 자료구조(예: 배열, 트리, 그래프 등)에 의존함.

탐색과 자료구조

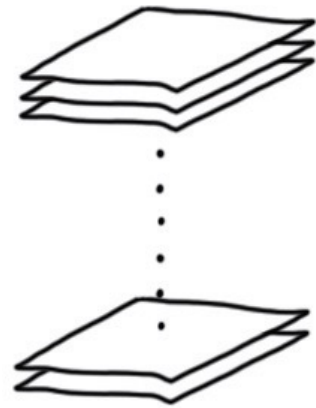
- 구조가 비교적 단순한 데이터를 대상으로 한 탐색 기법(예: 순차, 이진 탐색 등)은 주로 배열과 같이 비교적 단순한 자료구조에서 탐색 수행



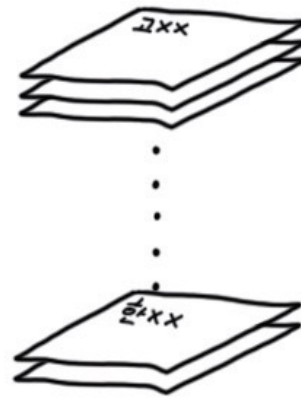
무작위 탐색(Random Search)



순차 탐색(Sequential or Linear Search)

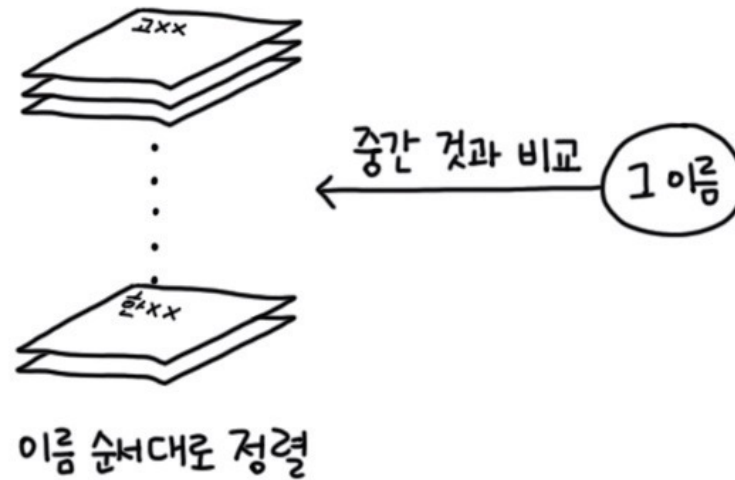


차례대로 배열



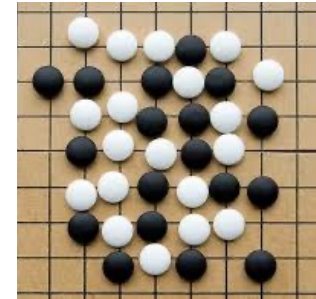
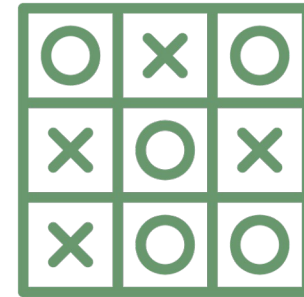
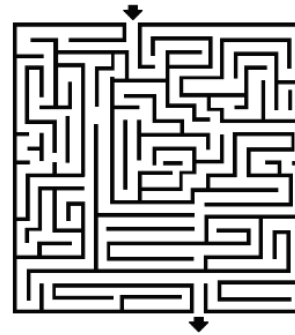
이름 순서대로 정렬

이진 탐색(Binary Search)



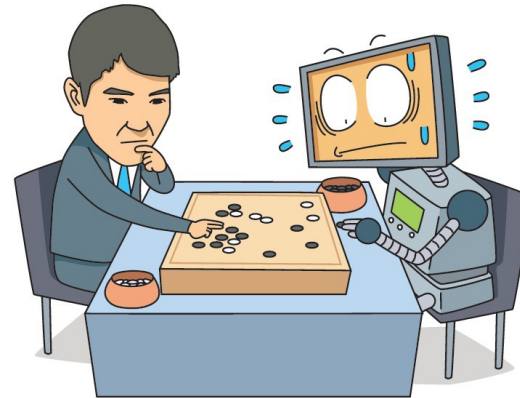
탐색 통한 문제 풀이(problem-solving)

- 고전적으로 탐색 알고리즘을 활용하는 인공지능 에이전트의 경우, 미로(길) 찾기 문제, 틱택토, 체스나 바둑 같은 복잡한 상태를 가지는 문제 풀이 (problem-solving) 상황에 활용됨.
- 이러한 문제 풀이 에이전트는 쉽게 목표를 달성할 수 없는 상황에서 적절한 미래 계획을 수립해야 함. 이와 같은 인공지능 에이전트를 문제 해결 에이전트(problem-solving agent)라고 부르며, 에이전트가 **목표 상태로의 경로를 형성하는 동작열 (sequence of actions)의 산출하는 계산 과정을 탐색(search)**이라고 부름.

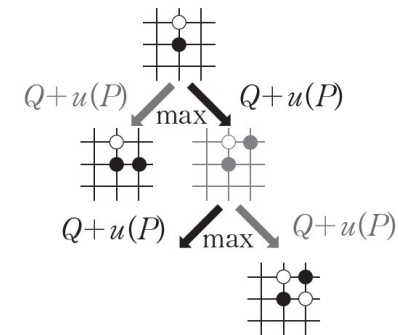


인공지능 탐색의 활용 분야

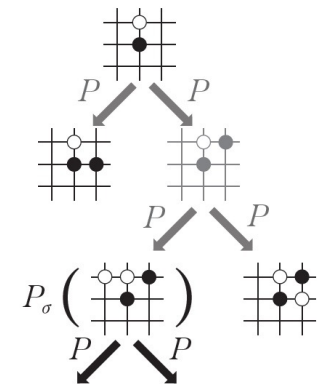
- 탐색 자체는 고전적인 인공지능 기법임에도 불구하고 최근 각광 받은 알파고(AlphaGo)에도 활용된 기법임
- 이 외에도 각종 탐색 기법은 현재 지도 및 네비게이션, 온라인 게임 등에 널리 활용되며, 최근 인공지능 기술에도 접목(예: 최적화 분야, seq2seq 모델 beam search 등)되어 기반 기술로 활용되고 있음



a Selection(선택)



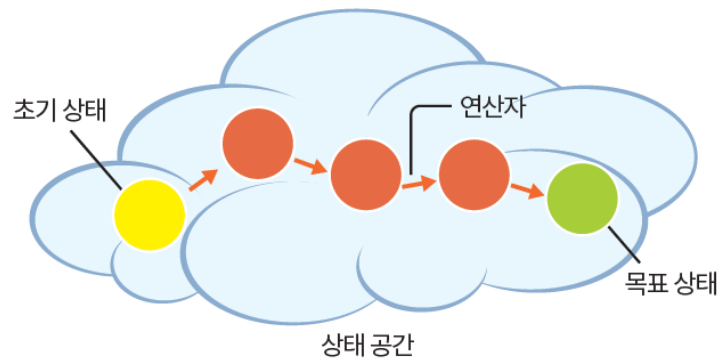
b Expansion(확장)



탐색 문제와 해

하나의 탐색 문제(search problem)는 아래와 같이 형식적으로 정의할 수 있음

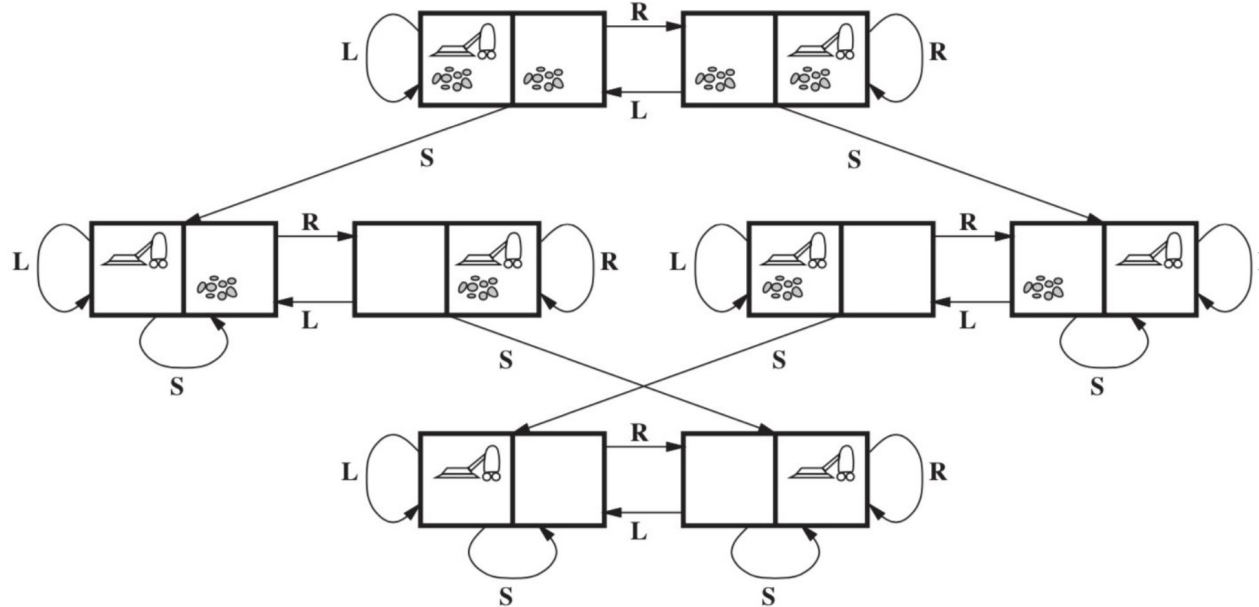
- **상태공간(state space)**: 환경의 가능한 상태(state)로 이루어진 집합
- **전이 모형(transition model)**: 각 동작이 하는 일을 서술하는 모형, 현재 상태에서 다음상태로 전이할 때의 규칙이나 함수
- **초기상태(initial state)**: 에이전트의 최초 상태
- **목표상태(goal state)**: 환경 내 추구하는 목표 상태(단일 혹은 여러 목표 집합이 될 수 있음)



탐색 통한 문제 풀이 과정

1. **목표 형식화(goal formulation):** 목표가 분명하면, 목적이 제한되고 에이전트가 고려할 동작 자체가 줄어들게 됨
2. **문제 형식화(problem formulation):** 목표 도달에 필요한 상태와 동작열을 고안. 목표 달성과 관련된 추상화된 모형 구축.
3. **탐색(search):** 목표 도달에 필요한 동작열을 발견할 때까지 앞서 구축한 모형 안에서 모의 실행하여, 해(solution)를 찾음 (단, 찾을 수 없는 경우에는 해가 없다는 결론에 이름).
4. **실행(execution):** 해답에 필요한 동작들을 실행

표준화된 장난감 문제: 예제 1. 진공청소기



- **상태(States):** 에이전트의 위치와 먼지의 위치를 의미함. 에이전트는 두 위치 중 하나에 있으며, 각각의 위치는 먼지가 있을 수도 있고 없을 수도 있음. 따라서 가능한 세계 상태는 2 (에이전트 위치: L/R) $\times$ 2 (먼지 유무: 있음/없음) $\times$ 2 (먼지 위치: L/R) = 8가지임.
- **초기 상태(Initial state):** 어떤 상태든 초기 상태로 지정할 수 있음.
- **동작(Actions):** 왼쪽(Left), 오른쪽(Right), 흡입(Suck).
- **전이 모형(Transition model):** 동작들은 예상되는 효과를 가지지만, 가장 왼쪽 칸에서 왼쪽으로 이동하거나, 가장 오른쪽 칸에서 오른쪽으로 이동하거나, 깨끗한 칸에서 흡입하는 경우에는 아무런 효과가 없음.
- **목표 상태(Goal state):** 모든 칸이 깨끗한지 확인함.

표준화된 장난감 문제: 예제 2. 8-Puzzle

7	2	4
5		6
8	3	1

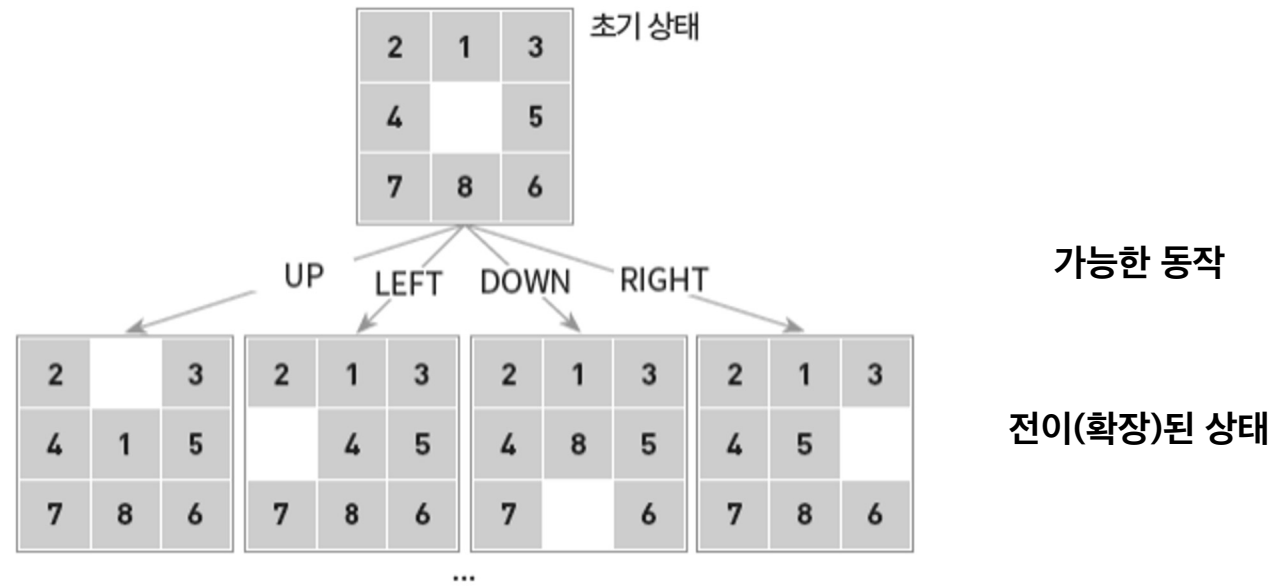
Start State

	1	2
3	4	5
6	7	8

Goal State

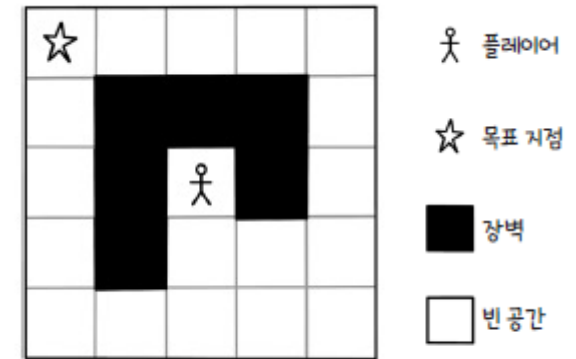
- **상태(States)**: 8개의 타일과 빈 칸의 위치를 의미함.
- **초기 상태(Initial state)**: 어떤 상태든 초기 상태로 지정할 수 있음.
- **동작(Actions)**: 왼쪽(Left), 오른쪽(Right), 위(Up), 아래(Down).
- **전이 모형(Transition model)**: 초기 상태에 왼쪽을 적용하면, 결과 상태는 5번 타일과 빈 칸의 위치가 바뀜.
- **목표 상태(Goal state)**: 현재 상태가 목표 구성과 일치하는지 확인함.

표준화된 장난감 문제: 예제 2. 8-Puzzle

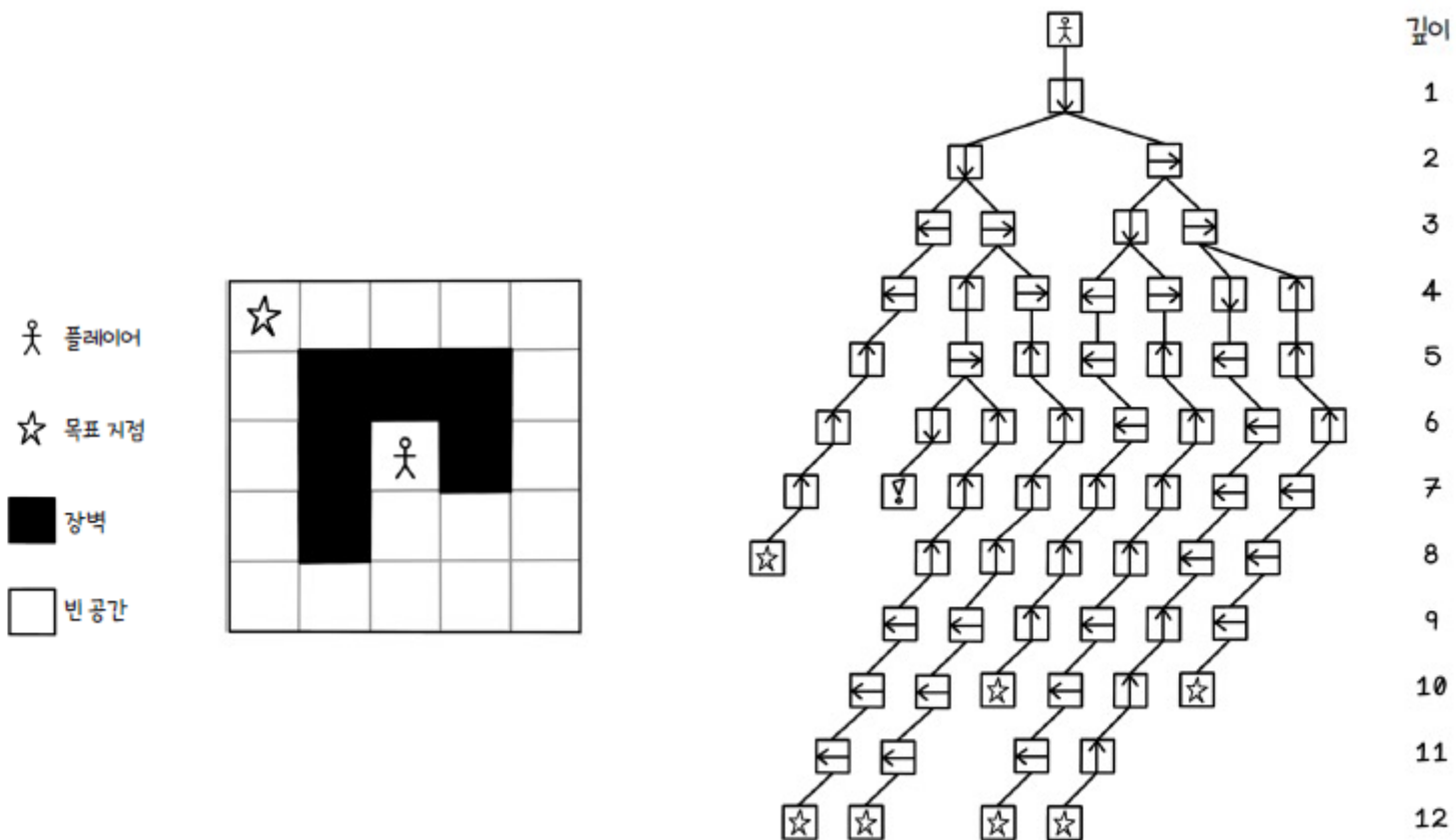


표준화된 장난감 문제: 예제 3. 미로찾기

- **상태(States):** 미로 내 에이전트(탐색자)의 위치와 미로의 구조를 의미함 (총 $x * y$ 개의 상태가 존재)
- **초기 상태(Initial state):** 에이전트가 미로의 시작 지점에 있는 상태
- **동작(Actions):** 에이전트는 상하좌우(North, South, East, West)로 이동할 수 있음 (하지만, 벽이 있는 방향으로 이동 불가)
- **전이 모형(Transition model):** 동작이 기대되는 효과를 가지며, 예를 들어 에이전트가 현재 위치에서 왼쪽(West)으로 이동하면, 에이전트는 해당 위치로 이동하게 됨. 그러나, 벽이 있는 경우 해당 동작은 무효.
- **목표 상태(Goal state):** 목표 지점은 미로 내의 특정 위치로, 해당 위치에 도달하는 것이 최종 목표.

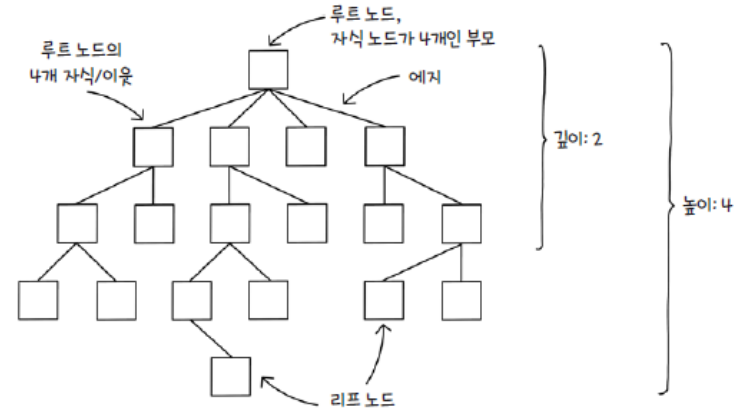
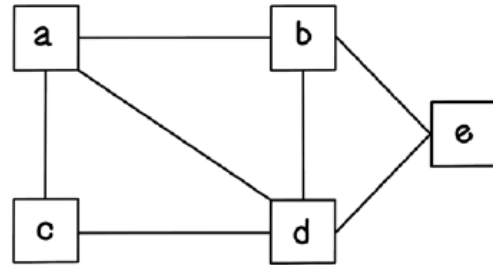


표준화된 장난감 문제: 예제 3. 미로찾기



탐색 상태 표현 위한 그래프와 트리 구조

- 각종 복잡한 문제에 대한 상태 공간을 수월하게 표현하기 위해, 트리 혹은 그래프 구조를 활용하는 경우가 대다수임.
- 트리의 경우, 그래프의 일종으로도 볼 수 있으며 비순환(acrylic)이자 방향성이 있는(directed) 그래프로 볼 수 있음.



문제 해결 성능 측정

- **완결성(completeness):** 문제에 해답이 있다면, 반드시 해답을 찾을 수 있는지 여부
- **최적성(optimality):** 가장 비용이 낮은 해답을 찾을 수 있는지 여부
- **시간 복잡도(time complexity):** 해답을 찾는데 걸리는 시간
- **공간 복잡도(space complexity):** 탐색을 수행하는 데 필요한 메모리의 양
 - b: 탐색 트리의 최대 분기 계수 (너비)
 - d: 목표 노드의 깊이
 - m: 트리의 최대 깊이

복잡도

- 빅 오 표기법(Big O Notation)을 통해 함수나 알고리즘의 복잡도를 설명할 수 있음.
- 입력 크기(n)가 커짐에 따라 필요한 연산 횟수를 모델링 함.
- 좋은 알고리즘은 입력 크기가 증가해도 연산 횟수가 제한적으로 늘어나거나 일정한 알고리즘.

예시

- Hello World 출력: $O(1)$
 - 목록에 있는 항목 출력: $O(n)$
 - 한 목록의 항목을 다른 목록의 항목과 비교 시: $O(n^2)$
-
- 계산 비용을 최소화하는 것이 문제를 빠르게 잘 해결하기 위한 지능형 알고리즘의 주요 목적 중 하나

